

Triển khai Nền tảng WisdomCloud — Dịch vụ Tra cứu Báo giá Cloud theo Mô hình JAMstack

Nguyễn Vũ 1,2, Nguyễn Lê Thanh Phong 2, Huỳnh Thanh Sơn 2

1 Người báo cáo chính 2 Wisdom Software

TP. Hồ Chí Minh, Tháng 06 năm 2026

Ngày 09 tháng 6 năm 2026

Tóm tắt nội dung

Báo cáo này trình bày toàn diện quá trình nghiên cứu, triển khai và vận hành nền tảng **WisdomCloud** — hệ thống tra cứu và so sánh báo giá dịch vụ Cloud (Cloud Server, Cloud Email, Cloud Security, Storage, Web Hosting, Data Center, Domain) từ nhiều nhà cung cấp tại Việt Nam. Dự án bao gồm bốn giai đoạn chính: (1) Thu thập và chuẩn hóa dữ liệu 175 dịch vụ Cloud từ 7 nhà cung cấp; (2) Xây dựng backend API với Node.js/Express phục vụ tìm kiếm, lọc, sắp xếp và phân trang; (3) Phát triển giao diện người dùng với React 18 + Vite hiện đại, trực quan; (4) Triển khai lên Vercel thông qua CI/CD tự động từ GitHub, bao gồm cấu hình experimentalServices và serverless function. Báo cáo tuân thủ các tiêu chuẩn REST API design và JSON data format trong thiết kế hệ thống.

1. Giới thiệu

1.1 Bối cảnh và Động lực

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ tại Việt Nam, nhu cầu thuê dịch vụ Cloud của các doanh nghiệp vừa và nhỏ ngày càng tăng cao. Các nhà cung cấp như CMC Cloud, FPT Cloud, Viettel Cloud, VNPT, vHost đều có bảng giá riêng với cấu hình và thông số kỹ thuật khác nhau, gây khó khăn cho khách hàng trong việc so sánh và lựa chọn. Dự án **WisdomCloud** được hình thành như một nỗ lực nghiên cứu và triển khai thực tiễn nền tảng tập trung hóa dữ liệu Cloud, tận dụng sức mạnh của kiến trúc JAMstack và nền tảng serverless Vercel. Mục tiêu cốt lõi của dự án không chỉ dừng lại ở việc xây dựng một giao diện web tra cứu đơn thuần, mà còn tập trung vào việc nghiên cứu sâu cơ chế vận hành của API serverless, định tuyến đa dịch vụ (multi-service routing) và quy trình CI/CD tự động — những thành phần được coi là nền tảng cho các ứng dụng web hiện đại. Thông qua nền tảng này, nhóm thực hiện hướng tới việc thiết lập một hệ thống tra cứu toàn diện, có khả năng tổng hợp và so sánh thông tin dịch vụ Cloud từ nhiều nguồn khác nhau.

1.2 Ý tưởng: Tập trung dữ liệu Cloud, So sánh thông minh

Ý tưởng cốt lõi của dự án xuất phát từ nhu cầu thực tế: thay vì phải truy cập lần lượt từng website của các nhà cung cấp Cloud để tra cứu giá, người dùng chỉ cần một điểm truy cập duy nhất. Mô hình WisdomCloud cho phép tạo ra một trung tâm dữ liệu tập trung, trong đó mọi thông tin dịch vụ, giá cả và thông số kỹ thuật đều được phản ánh đầy đủ. Cụ thể, quy trình xây dựng bao gồm:

- Thu thập dữ liệu:** Thông tin dịch vụ từ các nhà cung cấp được trích xuất từ file Excel và chuẩn hóa thành cấu trúc JSON thống nhất.
- Xây dựng API:** Backend Express cung cấp các REST endpoint cho phép tìm kiếm, lọc theo danh mục/nhà cung cấp/khoảng giá, sắp xếp và phân trang.
- Phát triển giao diện:** Frontend React với bố cục hiện đại, cho phép người dùng tương tác trực quan với dữ liệu.
- Tự động hóa triển khai:** Mỗi thay đổi trên GitHub được tự động build và deploy lên Vercel thông qua CI/CD.

1.3 Tiêu chuẩn hóa

Về mặt tiêu chuẩn hóa, hệ thống được xây dựng tuân thủ các nguyên tắc thiết kế REST API chuẩn, đảm bảo cấu trúc endpoint thống nhất và có thể hiểu được bởi các hệ thống khác. Các endpoint API được thiết kế theo convention `/api/v1/{resource}` với versioning rõ ràng. Bên cạnh đó, dữ liệu được chuẩn hóa theo cấu trúc JSON với các trường `id`, `name`, `slug`, `provider`, `category`, `price`, `unit`, `specs`, `description` — đảm bảo tính nhất quán và dễ dàng mở rộng. Sự kết hợp giữa REST API design và JSON data format cung cấp một khung sườn vững chắc cho việc trao đổi thông tin và thúc đẩy khả năng tương tác giữa Frontend và Backend.

1.4 Phạm vi Kỹ thuật

Phạm vi kỹ thuật của dự án tập trung vào ba trụ cột chính: kết nối đồng bộ Frontend-Backend, lưu trữ dữ liệu bền vững và triển khai tự động. Nhóm đã triển khai giải pháp kết nối giao diện người dùng dựa trên React với hệ thống Backend Express.js, cho phép quản lý dữ liệu JSON một cách linh hoạt. Một điểm nhấn quan trọng là việc áp dụng kiến trúc JAMstack, giúp tách biệt Frontend (SPA tĩnh) và Backend (API serverless), loại bỏ hoàn toàn gánh nặng quản lý hạ tầng server truyền thống. Cuối cùng, thông qua Vercel experimentalServices, dự án thiết lập quy trình triển khai tự động từ GitHub, cho phép cập nhật phiên bản mới chỉ trong vài phút.

2. Kiến trúc và Triển khai trên Môi trường Local

2.1 Thành phần Hệ thống và Công nghệ

```
graph TD
    subgraph "Client Layer"
        Browser["Trình duyệt Người dùng"]
    end

    subgraph "WisdomCloud Platform"
        subgraph "Frontend (React 18 + Vite)"
            REACT["Giao diện SPA<br/>Components: Header, Hero,<br/>SearchForm, SearchResults,<br/>ServiceDetail, Pagination, Footer"]
            AXIOS["Axios Client<br/>baseUrl: /api/v1"]
        end

        subgraph "Backend (Express.js)"
            EXPRESS["Express Server<br/>Node.js 20<br/>CORS enabled"]
            STATIC["Static File Server<br/>frontend/dist/"]
        end

        subgraph "Data Layer"
            DATA_JSON["data.json<br/>175 Cloud Services<br/>7 Categories<br/>7 Providers"]
        end

        subgraph "API Endpoints"
            SEARCH["GET /api/v1/cloud-services/search<br/>q, category, provider,<br/>minPrice, maxPrice,<br/>sort, page, size"]
            CATS["GET /api/v1/categories"]
            PROVS["GET /api/v1/providers"]
            DETAIL["GET /api/v1/cloud-services/detail<br/>?id=123 | ?slug=..."]
            STATS["GET /api/v1/cloud-services/stats"]
        end
    end
```

```

end

Browser -->|"http://localhost:3000"| REACT
Browser -->|"Axios GET /api/v1/..."| EXPRESS
REACT --> AXIOS
AXIOS --> SEARCH
AXIOS --> CATS
AXIOS --> PROVS
AXIOS --> DETAIL
AXIOS --> STATS
EXPRESS -->|"Đọc dữ liệu"| DATA_JSON
SEARCH --> DATA_JSON
CATS --> DATA_JSON
PROVS --> DATA_JSON
DETAIL --> DATA_JSON
STATS --> DATA_JSON
EXPRESS -->|"Static Files"| STATIC
STATIC --> Browser
REACT -->|"npm run dev<br/>Vite Dev Server"| Browser

```

Hình 1: Sơ đồ kiến trúc tổng thể của hệ thống WisdomCloud trên môi trường local, bao gồm các thành phần Frontend, Backend, Cơ sở dữ liệu và API endpoints.

Frontend: Thành phần giao diện người dùng được xây dựng bằng React 18 với Vite làm bundler. Quy trình triển khai bắt đầu bằng việc khởi tạo dự án với `npm create vite@latest`, sau đó cài đặt các gói phụ thuộc: `react`, `react-dom`, `axios`, và `@vitejs/plugin-react`. Kết quả là một giao diện web trực quan hoạt động tại địa chỉ `localhost:5173` (Vite dev server), cho phép người dùng tương tác với dữ liệu Cloud thông qua bố cục gồm: thanh Header với menu điều hướng; Hero section với domain search và thống kê ẩn tượng; SearchForm với các trường từ khóa, danh mục (dropdown), nhà cung cấp và khoảng giá; SearchResults hiển thị dạng card grid; ServiceDetail modal popup; Pagination và Footer.

Axios được sử dụng làm HTTP client với baseURL `/api/v1` (trên local). Cơ chế tự động chuyển đổi baseURL giúp frontend hoạt động trên cả môi trường local và production mà không cần thay đổi code:

```

const IS_VERCEL = import.meta.env.VITE_VERCEL === 'true';
const api = axios.create({
  baseURL: IS_VERCEL ? '/_/backend/api/v1' : '/api/v1',
  timeout: 5000,
});

```

Backend: Thay vì sử dụng các framework phức tạp, hệ thống sử dụng giải pháp tối ưu là Express.js — một web framework nhẹ cho Node.js. Server Express, chạy tại cổng 3000, đóng vai trò là nhà cung cấp API RESTful cốt lõi, cho phép thực hiện toàn bộ các thao tác tìm kiếm, lọc, sắp xếp và phân trang trên dữ liệu Cloud. Để đảm bảo tính tương tác mượt mà với Frontend chạy trên cổng 5173 (Vite) hoặc cổng 3000 (production build), middleware `cors()` đã được cấu hình nhằm khắc phục triệt để lỗi chặn tài nguyên từ nguồn khác (CORS).

Ngoài ra, trong quá trình phát triển, các endpoint API được kiểm thử trực tiếp thông qua Postman collection, giúp nhóm kiểm tra tính sẵn sàng và thử nghiệm các tham số truy vấn khác nhau.

Cơ sở dữ liệu: Để tối giản hạ tầng và chi phí vận hành, dự án sử dụng dữ liệu dạng JSON tĩnh thay vì cơ sở dữ liệu truyền thống. Tập `data.json` lưu trữ 175 dịch vụ Cloud với cấu trúc JSON phân cấp, hoàn toàn phù hợp với định

dạng dữ liệu của frontend, đồng thời cung cấp khả năng mở rộng linh hoạt và hiệu suất cao cho các thao tác tìm kiếm trong bộ nhớ. Mỗi service object có cấu trúc:

```
{
  "id": 1,
  "name": "C6 Small 1",
  "slug": "c6-small-1",
  "provider": "CMC Cloud",
  "category": "Cloud Server",
  "price": 725000.0,
  "unit": "vnd/tháng",
  "specs": "vCPU 1\nRAM 1 GB\nBăng thông nội bộ VPC: 500Mbps\nBăng thông quốc tế: 10 Mbps",
  "description": "Dịch vụ C6 Small 1 từ CMC Cloud - Cloud Server - 725,000đ/tháng"
}
```

Middleware: Axios đóng vai trò là middleware HTTP client quan trọng, thực hiện kết nối giữa Frontend và Backend. Thành phần này thu hẹp khoảng cách giao tiếp bằng cách tự động chuyển đổi baseUrl dựa trên biến môi trường, quản lý timeout và xử lý response/error một tập trung. Quy trình xử lý dữ liệu tuân theo mô hình request-response: đầu tiên frontend gửi request với các tham số tìm kiếm; backend xử lý và trả về JSON response; frontend nhận dữ liệu và render giao diện.

2.2 Quy trình Thực hiện

Cấu hình Hạ tầng: Quá trình thiết lập hạ tầng kỹ thuật yêu cầu sự tinh chỉnh chính xác để đảm bảo khả năng tương tác thông suốt giữa giao diện người dùng và máy chủ dịch vụ backend. Tập backend/server.js được cấu hình để phục vụ đồng thời API và file tĩnh từ thư mục frontend/dist/. Biến môi trường PORT cho phép tùy chỉnh cổng (mặc định 3000). Việc thay đổi này giúp khắc phục các lỗi "404 Not Found" phát sinh khi frontend cố gắng gọi các dịch vụ không tồn tại. Kết quả là máy chủ đã hoạt động ổn định như một điểm truy cập duy nhất (Single Point of Access) tại địa chỉ http://localhost:3000 .

Định tuyến Dữ liệu: Tập backend/server.js đã được xây dựng chi tiết nhằm điều phối quá trình ánh xạ thông tin. Hệ thống đăng ký 5 endpoint RESTful:

Endpoint	Method	Mô tả
/api/v1/cloud-services/search	GET	Tìm kiếm với tham số q, category, provider, minPrice, maxPrice, sort, page, size
/api/v1/categories	GET	Danh sách danh mục
/api/v1/providers	GET	Danh sách nhà cung cấp
/api/v1/cloud-services/detail	GET	Chi tiết dịch vụ
/api/v1/cloud-services/stats	GET	Thống kê

Endpoint /search xử lý các tham số truy vấn, lọc dữ liệu trong mảng, sắp xếp theo trường (name, provider, price, category) và thực hiện phân trang với kích thước tùy chỉnh (tối đa 100 kết quả/trang). Dữ liệu sau đó được trả về dưới dạng JSON với cấu trúc { content, page, size, totalElements, totalPages, last } .

3. Kết quả Triển khai trên Local

Giao diện Người dùng

Hệ thống đã triển khai thành công giao diện Web UI dựa trên React 18 + Vite, vận hành ổn định tại địa chỉ `http://localhost:3000` (hoặc `http://localhost:5173` ở chế độ dev). Bố cục giao diện được thiết kế tối ưu với cấu trúc trực quan, cho phép người quản trị tra cứu toàn diện hệ thống dịch vụ Cloud. Hero section đóng vai trò là điểm nhấn chính, hiển thị banner gradient với thông điệp "Giải Pháp Cloud Toàn Diện Cho Doanh Nghiệp", domain search nhanh, và các số liệu thống kê ấn tượng. Phần service grid trình bày 4 danh mục chính dưới dạng card. SearchForm cung cấp các bộ lọc đa dạng giúp người dùng dễ dàng tìm kiếm. Khu vực kết quả hiển thị danh sách dịch vụ dạng card grid với thông tin tên, nhà cung cấp, giá, thông số kỹ thuật và nút "Xem chi tiết".

(Hình 2: Giao diện Frontend của hệ thống WisdomCloud, cho phép người dùng tương tác với dữ liệu Cloud thông qua bố cục Hero, SearchForm, Results grid — ảnh chụp màn hình sẽ được cập nhật)

Khả năng Quản lý

Nền tảng hỗ trợ tìm kiếm toàn văn trên 4 trường (name, provider, specs, description) với khả năng kết hợp nhiều bộ lọc: danh mục (category dropdown), nhà cung cấp (provider), khoảng giá (minPrice/maxPrice). Người dùng có thể sắp xếp kết quả theo tên, nhà cung cấp, giá hoặc danh mục theo chiều tăng/giảm. Tính năng phân trang cho phép điều hướng qua nhiều trang kết quả với kích thước trang tùy chỉnh. Ngoài ra, modal ServiceDetail hiển thị thông tin chi tiết của từng dịch vụ bao gồm giá (định dạng tiền Việt Nam), thông số kỹ thuật, mô tả và nút liên hệ trực tiếp.

(Hình 3: Modal ServiceDetail hiển thị thông tin chi tiết dịch vụ Cloud — ảnh chụp màn hình sẽ được cập nhật)

Tích hợp Dữ liệu

Dự án đã thiết lập thành công nguồn dữ liệu với 175 dịch vụ Cloud được trích xuất từ file Excel và chuẩn hóa sang JSON. Các danh mục bao gồm:

Danh mục	Số lượng	Nhà cung cấp
Cloud Server	82	CMC Cloud, FPT, Viettel, VNPT, vHost
Cloud Email	27	CMC Cloud, FPT, Viettel, VNPT
Cloud Security	22	CMC Cloud, FPT, Viettel
Domain	8	—
Storage	16	CMC Cloud, FPT, Viettel
Web Hosting	17	CMC Cloud, FPT, vHost
Data Center	3	Viettel, VNPT

Bảng 1: Phân bố dịch vụ theo danh mục và nhà cung cấp — tổng cộng 175 dịch vụ từ 7 nhà cung cấp.

(Hình 4: Cấu trúc dữ liệu JSON của WisdomCloud — ảnh chụp màn hình sẽ được cập nhật)

Hệ thống API

Hệ thống cung cấp bộ API RESTful toàn diện với 5 endpoint chính. Endpoint `/api/v1/cloud-services/search` cho phép tìm kiếm linh hoạt với các tham số `q`, `category`, `provider`, `minPrice`, `maxPrice`, `sort`, `page`, `size`. Các endpoint `/categories`, `/providers`, `/stats` cung cấp dữ liệu tham chiếu. Endpoint `/cloud-`

`services/detail` hỗ trợ tra cứu chi tiết theo `id` hoặc `slug`. Tất cả các endpoint đều có xử lý lỗi đầu vào (validation) và trả về mã trạng thái HTTP phù hợp (200, 400, 404, 500).

(Hình 5: API endpoints của WisdomCloud được kiểm thử qua Postman — ảnh chụp màn hình sẽ được cập nhật)

Ví dụ request tìm kiếm:

```
GET /api/v1/cloud-services/search?q=cloud&minPrice=100000&page=0&size=5
```

Response:

```
{
  "content": [
    {
      "id": 1,
      "name": "C6 Small 1",
      "provider": "CMC Cloud",
      "category": "Cloud Server",
      "price": 725000.0,
      "specs": "vCPU 1\nRAM 1 GB\nBăng thông nội bộ VPC: 500Mbps\nBăng thông quốc tế: 10
Mbps"
    }
  ],
  "page": 0,
  "size": 5,
  "totalElements": 12,
  "totalPages": 3,
  "last": false
}
```

Listing 1: Ví dụ request và response của endpoint tìm kiếm dịch vụ Cloud.

4. Thảo luận

Giải quyết Vấn đề Kỹ thuật

Trong giai đoạn triển khai, nhóm đã đối mặt và xử lý triệt để ba thách thức kỹ thuật quan trọng.

Thứ nhất là **lỗi định tuyến Vercel**, phát sinh do Vercel chỉ hỗ trợ một endpoint cho serverless function tại thư mục `/api`. Để chạy đồng thời Frontend (SPA) và Backend (Express), nhóm đã áp dụng tính năng `experimentalServices` của Vercel, cho phép định nghĩa hai service riêng biệt với route prefix khác nhau: frontend tại `/` và backend tại `/_/backend`. Vấn đề được khắc phục bằng cách điều chỉnh cấu hình hạ tầng trong `vercel.json`.

Thứ hai là **lỗi API URL không đồng nhất**, xảy ra khi URL API trên Vercel (`/_/backend/api/v1`) khác với local (`/api/v1`). Giải pháp là sử dụng biến môi trường `VITE_VERCEL` kết hợp với logic kiểm tra trong Axios client để tự động chuyển đổi baseUrl.

Thứ ba là **đồng bộ dữ liệu**, khi dữ liệu được sử dụng bởi cả Express server (`backend/data.json`) và Vercel serverless function (`api/data.json`). Nhóm tạo hai bản sao và đồng bộ thủ công để tránh phụ thuộc cross-service.

Ưu điểm của Kiến trúc JAMstack

Việc áp dụng kiến trúc JAMstack là bước tiến quan trọng nhằm giải quyết bài toán chi phí vận hành và hiệu suất. JAMstack được lựa chọn nhờ các ưu điểm:

- **Hiệu suất cao:** Frontend là SPA tính được phục vụ qua CDN toàn cầu, giảm tải cho backend.
- **Chi phí thấp:** Không cần quản lý server riêng, Vercel cung cấp gói free với 100GB băng thông.
- **CI/CD tự động:** Mỗi lần `git push` lên GitHub, Vercel tự động build và deploy.
- **Mở rộng dễ dàng:** Khi cần thêm tính năng, chỉ cần thêm endpoint API mới.

Khả năng tương tác

Tính liên thông được thể hiện qua cơ chế API chuẩn RESTful, cho phép bất kỳ client nào (web, mobile, Postman) cũng có thể tương tác với dữ liệu. Postman collection được xây dựng với 8 request mẫu bao phủ tất cả các endpoint và trường hợp sử dụng:

- Search theo keyword
- Search theo category
- Search theo provider
- Search theo khoảng giá
- Search với phân trang
- Lấy danh sách categories
- Lấy danh sách providers
- Lấy thống kê

(Hình 6: Postman collection kiểm thử API WisdomCloud — ảnh chụp màn hình sẽ được cập nhật)

5. Triển khai lên Vercel

Sau khi hoàn thành giai đoạn phát triển và kiểm thử trên môi trường local, nhóm tiến hành triển khai hệ thống lên nền tảng Vercel. Quá trình này bao gồm nhiều bước từ tiếp nhận thông tin dự án, thiết lập kết nối GitHub, thực hành deploy các thành phần cho đến cấu hình routing và CI/CD.

5.1 Tiếp nhận Thông tin Vercel Project

Vercel cung cấp cho nhóm một nền tảng serverless để phục vụ việc triển khai và vận hành dự án WisdomCloud. Thông tin dự án bao gồm:

Thông số	Chi tiết
Nền tảng	Vercel (Serverless + Static Hosting)
Tên dự án	openclaw
Production URL	https://openclaw-eight-opal.vercel.app
GitHub repository	nvu28/product-search (SSH)
Phương thức CI/CD	Auto-deploy từ <code>git push origin main</code>
Mục đích	Deploy ứng dụng web Cloud Services

Bảng 2: Thông tin dự án Vercel.

Dự án này đóng vai trò là môi trường production cho hệ thống WisdomCloud, cho phép kiểm tra khả năng vận hành thực tế trước khi đưa vào sử dụng chính thức.

5.2 Thiết lập và Kết nối GitHub

Để quản lý mã nguồn và tự động hóa triển khai, nhóm sử dụng GitHub kết hợp với Vercel. Quy trình thiết lập kết nối bao gồm:

1. **Tạo repository trên GitHub:** Repository `product-search` được tạo với remote origin `git@github.com:nvu28/product-search.git`.
2. **Cấu hình SSH key:** SSH key `id_ed25519_product_search` được tạo và thêm vào GitHub để xác thực.
3. **Kết nối Vercel với GitHub:** Dự án Vercel được liên kết với repository GitHub thông qua Vercel Dashboard, cho phép CI/CD tự động.
4. **Cấu hình .gitignore:** Chặn các file không cần thiết: `node_modules/`, `dist/`, `*.xlsx`, `workflow/`, `.opencode/`.

Kết quả: Việc kết nối GitHub-Vercel đã được thiết lập thành công, cho phép nhóm push code và tự động deploy chỉ trong vài phút.

5.3 Thực hành Deploy các Thành phần

Trước khi triển khai hệ thống WisdomCloud chính thức lên Vercel, nhóm đã thực hiện các bước deploy từng thành phần để đảm bảo quy trình triển khai hoạt động trơn tru.

5.3.1 Cấu hình vercel.json

Tập `vercel.json` là trung tâm cấu hình của toàn bộ hệ thống:

```
{
  "buildCommand": "cd frontend && npm install && npm run build",
  "outputDirectory": "frontend/dist",
  "experimentalServices": {
    "frontend": {
      "root": "frontend",
      "routePrefix": "/",
      "framework": "vite"
    },
    "backend": {
      "root": "backend",
      "entrypoint": "server.js",
      "routePrefix": "/_/backend"
    }
  }
}
```

Listing 2: Cấu hình `vercel.json` với `experimentalServices` cho hai service Frontend và Backend.

5.3.2 Deploy Frontend (SPA tĩnh)

Frontend được build thành SPA tĩnh trong thư mục `frontend/dist/` thông qua lệnh `npm run build`. Vercel tự động phục vụ thư mục này tại route `/`. Quá trình build bao gồm:

- Cài đặt dependencies: `npm install`
- Build React app: `vite build`
- Output: `frontend/dist/` với các file HTML, JS, CSS đã được minify

5.3.3 Deploy Backend (Serverless Function)

Backend Express.js được Vercel chạy dưới dạng serverless function tại route `/_/backend`. Tập `server.js` được Vercel tự động phát hiện và chuyển đổi thành serverless function nhờ cấu hình `entrypoint` trong `experimentalServices`. Ngoài ra, một bản sao dữ liệu được tạo tại `api/data.json` và một serverless function phụ tại `api/search.js` để đảm bảo tính tương thích.

Cơ chế này cho phép backend:

- Tự động scale theo số lượng request
- Chỉ tính phí theo thời gian chạy (không tính phí khi idle)
- Tích hợp sẵn HTTPS, CDN và edge network

5.3.4 CI/CD Pipeline với GitHub

Quy trình CI/CD được thiết lập hoàn chỉnh:

```
Local Development → git add -A → git commit → git push origin main
  → Vercel nhận webhook từ GitHub
  → Build frontend (npm install + npm run build)
  → Deploy cả hai service (frontend /, backend /_/backend)
  → Production URL: https://openclaw-eight-opal.vercel.app
```

Biến môi trường `VITE_VERCEL=true` được cấu hình trên Vercel Dashboard để frontend biết đang chạy trên môi trường production và sử dụng baseUrl `/_/backend/api/v1`.

(Hình 7: Vercel Dashboard hiển thị trạng thái deploy thành công — ảnh chụp màn hình sẽ được cập nhật)

6. Nghiên cứu Kiến trúc Triển khai Chuẩn

Sau giai đoạn triển khai, nhóm nhận thấy rằng việc phát triển các ứng dụng web — đặc biệt là hệ thống tra cứu dịch vụ Cloud — cần phải tuân theo một kiến trúc chuẩn nhằm đảm bảo hiệu suất, khả năng mở rộng và tính khả dụng cao. Hiện tại, nhóm đang tích cực nghiên cứu ba hướng chính: kiến trúc JAMstack và Serverless, cơ chế cache và CDN, và mở rộng tính năng cho nền tảng.

6.1 Yêu cầu và Tiêu chuẩn khi Triển khai

Dựa trên nghiên cứu tài liệu chuyên sâu, việc triển khai mã nguồn lên môi trường production cần tuân thủ các yêu cầu:

- **Hiệu quả tài nguyên:** Hệ thống phải tối ưu hóa việc tiêu thụ tài nguyên (CPU, RAM), đặc biệt khi triển khai trên nền tảng serverless nơi chi phí tính theo thời gian chạy. Việc sử dụng SPA tĩnh (static site) được khuyến nghị vì tiêu thụ ít tài nguyên hơn so với server-side rendering truyền thống.
- **Độ trễ thấp:** Đối với ứng dụng tra cứu, yêu cầu quan trọng là độ trễ thấp (low latency) để đảm bảo phản hồi nhanh giữa giao diện và dữ liệu.
- **Tính khả dụng cao (High Availability):** Hệ thống cần có khả năng tự phục hồi và mở rộng (scalability) theo nhu cầu. Nền tảng Vercel cung cấp sẵn cơ chế auto-scaling cho serverless functions.
- **Tiêu chuẩn giao thức:**
 - Giao tiếp web/API: Sử dụng RESTful API với JSON format.
 - Bảo mật: HTTPS mặc định trên Vercel, CORS configuration cho phép frontend gọi API.

6.2 Kiến trúc Serverless dựa trên JAMstack

Các tài liệu đều hướng tới kiến trúc JAMstack kết hợp với Serverless:

- **Kiến trúc JAMstack:** Thay vì mô hình Monolith (nguyên khối), mã nguồn được chia thành Frontend (JavaScript) và Backend (API). Frontend là SPA tĩnh, Backend là các serverless function độc lập, dễ dàng phát triển, triển khai và mở rộng.
- **Mô hình Static Site + Serverless API:** Frontend được build thành file tĩnh (HTML, JS, CSS) và phục vụ qua CDN. API được triển khai dưới dạng serverless function, chỉ chạy khi có request.
- **Container Orchestration:** Trong tương lai, khi hệ thống mở rộng với nhiều dịch vụ hơn, nhóm có thể sử dụng Docker và Kubernetes để quản lý các microservices. Mỗi dịch vụ chạy trong một container riêng biệt và giao tiếp qua REST API.

6.3 Nghiên cứu về Caching và CDN

Nhóm đang nghiên cứu các giải pháp tối ưu hóa hiệu suất cho hệ thống WisdomCloud:

6.3.1 Tối ưu hóa với Edge Caching

Trong Vercel, Edge Caching cho phép lưu trữ response tại các edge node gần người dùng nhất. Các API response có thể được cache với `Cache-Control` header, giảm tải cho serverless function và cải thiện thời gian phản hồi. Ví dụ cấu hình cho endpoint categories (dữ liệu ít thay đổi):

```
res.setHeader('Cache-Control', 's-maxage=3600, stale-while-revalidate');
```

Listing 3: Ví dụ cấu hình cache header cho API ít thay đổi.

6.3.2 Tối ưu hóa với Incremental Static Regeneration

Đối với các trang danh mục và provider, Vercel hỗ trợ Incremental Static Regeneration (ISR), cho phép tạo trang tĩnh nhưng vẫn cập nhật dữ liệu định kỳ mà không cần rebuild toàn bộ ứng dụng.

6.4 Nghiên cứu về Mở rộng Tính năng

Nhóm đang nghiên cứu các tính năng mới cho WisdomCloud dựa trên mô hình của các nền tảng thương mại điện tử Cloud:

- **So sánh giá:** Cho phép người dùng chọn nhiều dịch vụ và so sánh thông số kỹ thuật, giá cả cạnh nhau.
- **Trang nhà cung cấp:** Trang tổng quan riêng cho từng nhà cung cấp (CMC Cloud, FPT, Viettel...) hiển thị tất cả dịch vụ của họ.
- **Tìm kiếm nâng cao:** Lọc theo cấu hình (RAM, vCPU, Storage), hỗ trợ người dùng tìm đúng dịch vụ phù hợp nhu cầu.
- **Tự động cập nhật dữ liệu:** Sử dụng web scraping để tự động đồng bộ giá và thông số từ website các nhà cung cấp.

6.5 Kế hoạch Triển khai Tiếp theo

Dựa trên kết quả nghiên cứu, nhóm đề xuất kế hoạch triển khai các tính năng mới cho WisdomCloud theo các bước sau:

1. **Bước 1 — So sánh giá:** Thêm endpoint API cho phép so sánh nhiều dịch vụ cùng lúc, xây dựng giao diện so sánh dạng bảng.
2. **Bước 2 — Trang nhà cung cấp:** Tạo trang riêng cho mỗi provider, hiển thị danh sách dịch vụ + thông tin nhà cung cấp.
3. **Bước 3 — Tìm kiếm nâng cao:** Thêm bộ lọc theo cấu hình (RAM, vCPU, Storage) vào SearchForm.
4. **Bước 4 — Web Scraping:** Xây dựng script Python tự động thu thập dữ liệu từ website các nhà cung cấp, cập nhật JSON.
5. **Bước 5 — Monitoring:** Tích hợp Vercel Analytics và Sentry để giám sát hiệu suất và lỗi.

7. Kết luận

Dự án **WisdomCloud** đã hoàn thành thành công ba giai đoạn quan trọng:

- Giai đoạn Nghiên cứu:** Nhóm đã nghiên cứu và nắm vững ý tưởng về tập trung hóa dữ liệu Cloud, hiểu rõ cơ chế vận hành của REST API, JAMstack và các tiêu chuẩn thiết kế hệ thống web hiện đại.
- Giai đoạn Triển khai Local:** Hệ thống đã được triển khai thành công trên môi trường local với đầy đủ các thành phần: giao diện React hoạt động tại cổng 3000, backend Express cung cấp 5 API RESTful, cơ sở dữ liệu JSON với 175 dịch vụ Cloud, và Postman collection kiểm thử API.
- Giai đoạn Triển khai Production:** Nhóm đã thiết lập kết nối thành công với Vercel thông qua GitHub, cấu hình experimentalServices cho frontend và backend, thiết lập CI/CD tự động và hiện đang nghiên cứu chuyên sâu về kiến trúc JAMstack, Edge Caching và mở rộng tính năng để chuẩn bị cho các phiên bản tiếp theo của WisdomCloud.

Hướng phát triển tiếp theo tập trung vào việc hoàn thiện các tính năng so sánh giá, trang nhà cung cấp, tìm kiếm nâng cao và tự động hóa cập nhật dữ liệu — đảm bảo WisdomCloud vận hành liên tục, ổn định và có khả năng mở rộng trong môi trường sản xuất thực tế.

Lời cảm ơn

Báo cáo này được thực hiện bởi đội ngũ kỹ thuật tại Wisdom Software. Nhóm dự án bao gồm: Nguyễn Vũ (Người báo cáo chính), Nguyễn Lê Thanh Phong và Huỳnh Thanh Sơn. Nhóm xin gửi lời cảm ơn đặc biệt đến anh Đặng Văn Phúc Nghĩa đã hướng dẫn trực tiếp về quy trình deploy với Docker và server. Nghiên cứu được hoàn thành vào tháng 06 năm 2026 tại TP. Hồ Chí Minh.